Covariant Derivatives and Connections in the $\mathcal{UD}$ Calculus

Revised Edition

Brian Beckman

Jan 2026

Abstract

In this installment of the $\mathcal{UD}$ series, we address a central issue of differential geometry: how to take a derivative in (or on) a curved manifold without breaking coordinate invariance. Ordinary partial differentiation is not tensorial, producing garbage terms that don't transform properly. The solution is the ***covariant derivative*** and its correction term, the ***connection coefficient*** Γ.

We begin with the non-tensorial transformation law of Γ. Then, applying the constraints of metric compatibility ($\nabla g = 0$) and torsion freedom (symmetry on lower indices of Γ), we derive the particular connection coefficients needed for general relativity, the ***Christoffel Symbols***. This derivation specializes differential geometry to Riemannian geometry as needed for the general relativistic theory of gravitation. Step-by-step derivation in $\mathcal{UD}$ automates tiresome and risky algebraic operations for high assurance in the results.

At this early stage of development, we expose $\mathcal{UD}$ as a script rather than hiding details in a package. This glass-box approach delivers better pedagogy and incremental validation.

Preface to the Revised Edition

In this revised edition, I invert the relationship between comments and code. I break up the blocks of code-with-essential-comments from the first edition into alternating prose and code cells, as is usual with notebooks. The motivation for the original style was to promote the script versions, but I have since learned that the .wl script format is more flexible than I had originally assumed: it can host notebook-formatted comments—Section, Item, and so on—not just ASCII/Unicode comments. The result is that script versions of this notebook can follow the notebook format very closely.

I also take the opportunity to improve the presentation in some trivial, non-material ways.

- *A .wl script version of code for this notebook, minus the prose, is available at https://github.com/rebcabin/RelativityToolkit/blob/main/covariant_derivative_revised.wl.*

Audience

This series of notebooks is targeted at readers with an undergraduate's STEM education—engineering, computer science, mathematics. Exposure to introductory courses in Classical Mechanics (e.g., Goldstein), Electrodynamics (e.g., Jackson), and Relativity (e.g., Schutz) will help.

- *Except for MTW, I do not put standard textbooks in the References section, but offer clickable (non-affiliate!) links instead in the prose of this notebook.*

I also invite readers to "think like a physicist," that is, to demand conceptual cogency, but not always full mathematical rigour.

Transparent Implementation

In developing $\mathcal{UD}$, we made a conscious decision, for the time being, to expose the implementation as a glass-box **script** rather than to hide it in a Package, prioritizing transparency over encapsulation at this stage of development. When the toolkit is finalized in the future, we'll address encapsulating it in a Package.

Let's load $\mathcal{UD}$'s implementation, the RelativityToolkit, directly into your notebook context.

Initialize the Relativity Toolkit

The code is hosted on GitHub. The implementation, tests, and visualizations are not explained in this notebook. Explanations appear in an earlier notebook. But we can run them here, for assurance.

The toolkit is currently implemented as a Script rather than as a package for development speed and for transparency. That means that all symbols will be loaded into the Global namespace, including internal variables in Module constructs. This minor inconvenience will be mitigated in a future version.

Optional: Quit for a Fresh Kernel

- *Note, if you call* `Quit` *in a notebook, evaluation may hang. If you want to "Evaluate Notebook," mark the following two cells "Evaluatable" in the Cell→Property menu, call* `Quit` *and* `FrontEndTokenExecute` *once each, then comment out the cells, or mark them unevaluatable again. Alternatively, you can quit the kernel via the Evaluation menu (last item). At that point, you can "Evaluate Notebook" in a clean environment. Otherwise, you can evaluate cells in this notebook individually.*

In[•]:=
```
Quit[];
```

In[•]:=
```
FrontEndTokenExecute["DeleteGeneratedCells"]
```

Set Download Loci and Load Main Script

In[•]:=
```
RTbranch = "main";
RTbase = "https://raw.githubusercontent.com/rebcabin/RelativityToolkit/" <>
    RTbranch <> "/";
RTbase = "/Users/brian/Documents/GitHub/RelativityToolkit/";
RTbust = "?t=" <> ToString[Round[AbsoluteTime[]]];
RTbust = "";
(*freshen download*)
RTrules = RTbase <> "RelativityToolkit.wl" <> RTbust;
RTtests = RTbase <> "RelativityToolkit_RegressionTests.wl" <> RTbust;
RTviz = RTbase <> "RelativityToolkit_VisualShowcase.wl" <> RTbust;
Get[RTrules];
```

» Relativity Toolkit version : 1.8.0

» Relativity Connection Symbol : Γ

Optional: See all Tests and Results from Prior Works

Make any of the following cells evaluatable if you wish. They're unevaluatable by default to keep this notebook trim.

```
Get[RTtests];
```

In[•]:=
```
Get[RTviz](* no semicolon, on purpose *)
```

In[•]:=
```
Get[RTbase <> "covariant_derivative.wl"]
```

In[•]:=
```
Get[RTbase <> "law_of_gamma.wl"]
```

```
Get[RTbase <> "connections_to_christoffels.wl"]
```

```
Get[RTbase <> "calculate_riemann.wl"]
```

Optional: Configure Relativity Connection Symbol

To properly compose covariant derivatives, $\mathcal{UD}$ must know the user's reserved symbols for non-tensorial objects like connection coefficients. In gravitational physics, these force-producing connections are called Γ (capital gamma). Other theories like electrodynamics and Yang-Mills that can benefit from $\mathcal{UD}$ use other connection symbols.

We're going with Γ for this relativity/gravitation notebook, it's the default, so the following cell is optional.

In[•]:=
```
SetConnection[Γ];
```

```
Relativity Engine: Connection set to Γ
```

Need for Covariant Derivative

We Want a Derivative!

Consider a curve with invertible, continuous, differentiable coordinate functions $x^\alpha(\lambda)$ along the curve, and tangent (contravariant) vectors $u^\alpha \equiv dx^\alpha / d\lambda$. Also consider a (contravariant) vector field $A^\alpha\left(x^\beta(\lambda)\right)$ defined in neighborhoods around every point $x^\alpha(\lambda)$ along the curve. We'd like a "derivative" of the vector field, schematically:

$$\frac{dA^\alpha}{dx^\beta} \stackrel{?}{\equiv} \frac{A^\alpha\left(x^\beta + dx^\beta\right) - A^\alpha\left(x^\beta\right)}{dx^\beta} \tag{1}$$

- *We'll have a better, "cool" notation for $\frac{dA^\alpha}{dx^\beta}$ soon enough. For now, read it as "the derivative we're trying to define."*

Seems a reasonable thing to ask: "How much does component α of A change when I change coordinate β a little?"

However, Equation 1 is not a cogent definition of derivative on a manifold. We cannot subtract vectors at different points, not even infinitesimally different points. The coordinate axes at $\left(x^\beta + dx^\beta\right)$ don't point in the same directions as the coordinate axes at x^β because the manifold bends and twists away between the two points! The components of A are not commensurable at the two points because the two vectors are in **different tangent spaces**. Subtracting components as prescribed by Equation 1, numerically, scrambles information from other directions.

Conundrum of Parallel Transport

To answer this conundrum, we need backwards ***parallel transport***, to move the vector $A^\alpha\left(x^\beta + dx^\beta\right)$ smoothly and with a ***provable minimum*** of bending and twisting, back from $x^\beta + dx^\beta$ to the point x^β, so we can measure the components of the transported $A^\alpha\left(x^\beta + dx^\beta\right)$ on the coordinate axes at point x^β and then subtract the numbers.

$$\frac{dA^\alpha}{dx^\beta} \stackrel{?}{\equiv} \frac{\text{ParallelTransport}\left(A^\alpha\left(x^\beta + dx^\beta\right),\ \text{to } x^\beta\right) - A^\alpha\left(x^\beta\right)}{dx^\beta} \tag{2}$$

Conundrum of Coordinate-Independence

Now, with subtraction conceptually solved, how will that difference survive a coordinate transformation? How can I find out, when I **change coordinates from x^β to $x^{\beta'}$ at the same point**, how much component $\alpha \to \alpha'$ of A changes when I change coordinate $\beta \to \beta'$? That is, what's the value of $\frac{dA^{\alpha'}}{dx^{\beta'}}$ when all I know is $\frac{dA^\alpha}{dx^\beta}$?

To answer *this* conundrum, we insist, as always, that the ***intrinsic*** geometry and physics **not** depend on choice of coordinates. But what does *intrinsic* mean when all we have are numerical components whose values depend on choice of coordinates?

Intrinsic Means Tensorial

Tensorial quantities encapsulate both intrinsic and coordinate-dependent information. If our desired, derivative-like quantities $\frac{dA^\alpha}{dx^\beta}$ transform like a (1, 1) tensor when we change the basis vectors at point β, we'll capture both:

- *intrinsic* geometric and physical properties of a tensor
- *extrinsic* artifacts arising from choice of coordinates
  - Think of changing from Cartesian to spherical coordinates in flat Euclidean 3-space: the numbers change but the geometry and physics don't.

Plus, we will be able to separate intrinsic (coordinate-dependent) from extrinsic (coordinate-independent) quantities cleanly.

- *TODO: How?*
- *The first couple of chapters of Lovelock & Rund address this issue of intrinsic, geometry-dependent effects versus extrinsic, coordinate-dependent effects rigorously. Thinking like physicists, let's be satisfied with the conceptual description above for now.*

Tensorial and Linear Requirements

Agree that the components of our desired derivative, $\frac{dA^\alpha}{dx^\beta}$, transform between coordinate frames in exactly the way components of any other (1, 1) tensor—with one up and one down index—would change. These components are projections of the vector A on the β basis vectors of the coordinate system x^β, and measure both intrinsic and extrinsic information.

It turns out that simply demanding that our new derivative be **tensorial and linear** suffices to solve all the conundrums above, including optimality.

- *TODO: This claim of optimality may not be good enough, even for a physicist otherwise happy with mathematical hand-waving. Misner, Thorne, and Wheeler [MTW] devote many chapters (8-15) to a more careful, physicist's level of understanding, including demonstrations that the covariant derivative performs parallel transport! Those are missing pieces in this notebook.*
- *A mathematician will demand rigorous proofs. For those, see the many mathematical texts on differential geometry, perhaps starting with do Carmo's Differential Geometry of Curves and Surfaces. Other texts such as Tristan Needham, Visual Differential Geometry and Forms, and Michael Spivak's 5-volume Comprehensive Introduction to Differential Geometry are recommended.*

The Partials are Not Tensors

First, let's check that $A^\alpha{}_{,\beta}$ is not tensorial. It's not enough for our desired derivative, but we'll figure out how to correct it.

- *The comma notation, $A^{\alpha}{}_{,\beta}$, is short for $\frac{\partial A^{\alpha}}{\partial x^{\beta}}$, also written $\partial_{\beta} A^{\alpha}$. Though we don't yet know how to define $\frac{dA^{\alpha}}{dx^{\beta}}$, $\frac{\partial A^{\alpha}}{\partial x^{\beta}}$ is just good-ol' partial derivative from ordinary calculus, a mechanical, symbolical calculation that Wolfram automates very well.*
- ***𝒰𝒟 embraces the comma notation** as you can see from the Visual Showcase gallery above.*

Let's transform bases and calculate $A^{\alpha'}$, contravarian components in basis $x^{\alpha'}$, from A^{α}, contravariant components in basis x^{α}. *A* transforms as follows:

$$A^{\alpha'} \equiv \frac{\partial x^{\alpha'}}{\partial x^{\alpha}} A^{\alpha} \tag{3}$$

- *A mnemonic for "contravariant:" let $\frac{\partial x^{\alpha'}}{\partial x^{\alpha}}$ measure feet (primed) per inch (unprimed), that is (1/12). Any unprimed contravariant vector A^{α} is, mnemonically, like a velocity in inches per second, and its primed partner $A^{\alpha'}$ is, mnemonically, like a velocity in feet per second. Multiply A^{α} (inches per second) by $\frac{\partial x^{\alpha'}}{\partial x^{\alpha}}$ (feet per inch, 1/12) to get $A^{\alpha'}$ (feet per second), i.e., $A^{\alpha'} = \frac{\partial x^{\alpha'}}{\partial x^{\alpha}} A^{\alpha}$. The same rule goes for any kind of coordinate change, curvilinear, non-orthogonal, whatever: Contravariant? Multiply by $\frac{\partial x^{\alpha'}}{\partial x^{\alpha}}$.*
- *Another mnemonic: the unprimed up index of A^{α} matches* contrariwise *the unprimed down index of $\frac{\partial x^{\alpha'}}{\partial x^{\alpha}}$, given that an up index in the denominator ∂x^{α} of $\frac{\partial x^{\alpha'}}{\partial x^{\alpha}}$ is a down index of the whole $\frac{\partial x^{\alpha'}}{\partial x^{\alpha}}$. 𝒰𝒟 automates this **valence check**.*
- *The quantities $\frac{\partial x^{\alpha'}}{\partial x^{\alpha}}$ are the **Jacobian of the coordinate transformation** from x^{α} to $x^{\alpha'}$, sometimes written (totally ambiguously!) as J, with $\frac{\partial x^{\alpha}}{\partial x^{\alpha'}}$ being J^{-1}. In manual calculations, one must constantly check the "direction" of Jacobian every time; it's a fertile source of error. **𝒰𝒟 won't let you make this mistake!***
- *Covariant tensors transform with the matching index co-wise in the numerator of the Jacobian. The mnemonical example is a gradient, say degrees of temperature per foot or inch.*

Now calculate partial derivatives with respect to the primed frame:

$$A^{\alpha'}{}_{,\beta'} \equiv \frac{\partial}{\partial x^{\beta'}} \left(\frac{\partial x^{\alpha'}}{\partial x^{\alpha}} A^{\alpha} \right) \tag{4}$$

- *This notation is different from Lovelock & Rund, who write $\overline{x}^{\alpha}$ for the transformation functions, with Greek indices that have no primes. However, we will not be confused: $x^{\alpha'}$ is short for $x^{\alpha'}(x^{\beta})$, four functions of four arguments each in relativistic physics, N functions of N arguments more generally. These functions express new coordinates in terms of old. The prime isn't really on the index, but on the pair of literal x and the index. We write it on the index only for ease of typesetting. Also be reminded that the partial derivatives with respect to independent variables x^{β} are mechanical, symbolical operations.*

Calculus Via 𝒰𝒟

Let's have 𝒰𝒟 do this bit of calculus.

We'll need Wolfram-friendly notation for primed indices. α' won't do, except for display, because Wolfram interprets the prime as a derivative. We'll just write **ap** and **bp** for the primed indices α' and β'

and a pretty rule for display.

```
In[•]:= ClearAll[ap, bp, μ, pretty]; pretty = {ap → α', bp → β', mu → μ};
```

Define the Input

Express the partial via $\mathcal{UD}$'s inert `Partials` head:

```
In[•]:= Partials[A[𝒰[ap]], x[𝒰[bp]]] /. pretty
```

```
Out[•]=
```

$$A^{\alpha'}{}_{,\beta'}$$

Write $A^{\alpha'}$ as Transformed from A^{α}

Via $\mathcal{UD}$'s `robustTransformRules`, write $A^{\alpha'}$ as the Jacobian times A^{α}. The trailing `/.pretty//TensorForm` are for display only

```
In[•]:= Echo[(inputTerm = Partials[A[𝒰[ap]] /. robustTransformRules, x[𝒰[bp]]]) /. pretty //
          TensorForm,
        "∂β'(A^α') ≡ A^α',β' ="];
```

» $\partial_{\beta'}(A^{\alpha'}) \equiv A^{\alpha'}{}_{,\beta'} = \left(\left(A^{\lambda}\right)\frac{\partial x^{\alpha'}}{\partial x^{\lambda}}\right)_{,\beta'}$

Expand Derivatives

Apply Leibniz (product) and chain rules repeatedly until stable:

```
In[•]:= Echo[(expandedTerm = ExpandDerivatives[inputTerm]) /. pretty // TensorForm,
        "A^α',β' ="];
```

» $A^{\alpha'}{}_{,\beta'} = \left(A^{\lambda}\right)\frac{\partial^2 x^{\alpha'}}{\partial x^{\beta'}\partial x^{\lambda}} + \left(A^{\lambda}{}_{,\beta'}\right)\frac{\partial x^{\alpha'}}{\partial x^{\lambda}}$

Definition of Covariant Derivative

Partials of the Transform ≠ Transform of the Partials

The partial derivative $A^{\alpha}{}_{,\beta}$ is not tensorial, else it would transform like $\frac{\partial x^{\alpha'}}{\partial x^{\alpha}}\frac{\partial x^{\beta}}{\partial x^{\beta'}}A^{\alpha}{}_{,\beta}$. When we differentiate $A^{\alpha'}$, we get

$$\frac{\partial}{\partial x^{\beta'}}\left(A^{\alpha'} \equiv A^{\mu}\frac{\partial x^{\alpha'}}{\partial x^{\mu}}\right) \equiv A^{\alpha'}{}_{,\beta'} \equiv \left(A^{\mu}\frac{\partial x^{\alpha'}}{\partial x^{\mu}}\right)_{,\beta'} \tag{5}$$

we don't get the transform $\left(\frac{\partial x^{\alpha}}{\partial x^{\alpha'}}\frac{\partial x^{\beta'}}{\partial x^{\beta}}A^{\alpha'}{}_{,\beta'}\right)$ of the derivative $A^{\alpha'}{}_{,\beta'}$, but two terms that are difficult to interpret: $\left(A^{\lambda}\frac{\partial^2 x^{\alpha'}}{\partial x^{\beta'}\partial x^{\lambda}}\right)$ and $\left(A^{\lambda}{}_{,\beta'}\frac{\partial x^{\alpha'}}{\partial x^{\lambda}}\right)$.

These terms have factors, $\frac{\partial^2 x^{\alpha'}}{\partial x^{\beta'}\partial x^{\lambda}}$ and $A^{\lambda}{}_{,\beta'}$ respectively, that are calculated in both coordinate systems,

and that's disturbing. We can extract one Jacobian factor of $\frac{\partial x^\beta}{\partial x^{\beta'}}$, but that's it:

$$A^{\alpha'}{}_{,\beta'} = A^\lambda \left(\frac{\partial^2 x^{\alpha'}}{\partial x^{\beta'} \partial x^\lambda} \right) + \left(A^\lambda{}_{,\beta'} \frac{\partial x^{\alpha'}}{\partial x^\lambda} \right) =$$
$$A^\lambda \frac{\partial x^\beta}{\partial x^{\beta'}} \left(\frac{\partial^2 x^{\alpha'}}{\partial x^\beta \partial x^\lambda} \right) + \left(A^\lambda{}_{,\beta} \frac{\partial x^\beta}{\partial x^{\beta'}} \right) \frac{\partial x^{\alpha'}}{\partial x^\lambda} = \frac{\partial x^\beta}{\partial x^{\beta'}} \left(A^\lambda \frac{\partial^2 x^{\alpha'}}{\partial x^\beta \partial x^\lambda} + A^\lambda{}_{,\beta} \frac{\partial x^{\alpha'}}{\partial x^\lambda} \right) \tag{6}$$

The quantity $\frac{\partial x^\beta}{\partial x^{\beta'}} \left(A^\lambda \frac{\partial^2 x^{\alpha'}}{\partial x^\beta \partial x^\lambda} + A^\lambda{}_{,\beta} \frac{\partial x^{\alpha'}}{\partial x^\lambda} \right)$ is again not tensorial because it has a free index α'. Worse, it depends on both coordinate systems, the primed and the unprimed, and we can't have that! Worse still, $\frac{\partial^2 x^{\alpha'}}{\partial x^\beta \partial x^\lambda}$ the ***Hessian*** of the coordinate transformation, is not a geometrical object, but rather an algebraic object. It can't itself be transformed. It depends on particular choices of two coordinate systems. It's of no help.

The Way Out: Covariant Derivative

There's a way out: insist on a certain form.

Write the tensorial derivative we want, $\frac{dA^\alpha}{dx^\beta}$, as the non-tensorial partial derivative $A^\alpha{}_{,\beta}$ plus a correction.

Demand that the correction be linear in A^α and that the sum—partial derivative plus correction—transform tensorially. Defining the ***semicolon notation*** for this "corrected derivative" $A^\alpha{}_{;\beta}$

$$A^\alpha{}_{;\beta} \equiv A^\alpha{}_{,\beta} + \Gamma^\alpha{}_{\beta\mu} A^\mu \tag{7}$$

require that it transform as follows:

$$A^{\alpha'}{}_{;\beta'} = \frac{\partial x^{\alpha'}}{\partial x^\alpha} \frac{\partial x^\beta}{\partial x^{\beta'}} A^\alpha{}_{;\beta} \tag{8}$$

then solve for $\Gamma^\alpha{}_{\beta\mu}$. Notice contraction on the third index: $\Gamma^\alpha{}_{\beta\mu} A^\mu$.

$A^\alpha{}_{;\beta}$ is the ***covariant derivative***.

- *Despite the name, the covariant derivative is* contravariant *in α and* covariant *in β. The word "covariant" here does not have its usual meaning, but we're stuck with this historical nomenclature.*
- *The form of a term plus a correction is ubiquitous in linear theories such as that for the Kalman filter (see my notebook Kalman filtering as a functional fold).*
- *The contravariant vector A is tensorial, measuring intrinsic properties of A. Γ is not tensorial, mixing in extrinsic information about the coordinate system. The product A.Γ is tensorial again.*
- *If we do our job right (proving tensoriality and linearity), $A^\alpha{}_{;\beta}$ becomes the promised "cool notation" for $\frac{dA^\alpha}{dx^\beta}$, the derivative we want, the covariant derivative.*
- *One may also write $\partial_\beta A^\alpha$* for $A^\alpha{}_{,\beta}$ and *$\nabla_\beta A^\alpha$ for $A^\alpha{}_{;\beta}$. These forms frequently appear in the literature.*
- *TODO: Why must the correction $\Gamma^\alpha{}_{\beta\mu} A^\mu$ be linear in A?*
- *SPOILER: We want our new derivative ∇ (aka the semicolon) to behave like any other derivative. That means it must satisfy two properties:*

- *Linearity:* $\nabla_\mu (A^\nu + B^\nu) = \nabla_\mu A^\nu + \nabla_\mu B^\nu$
- *Leibniz Rule:* $\nabla_\mu (f A^\nu) = A^\nu \partial_\mu f + f \nabla_\mu A^\nu$

If we assume the general form of derivative plus a correction, $\nabla_\mu A^\nu = \partial_\mu A^\nu + C^\nu(A, \partial A, \partial^2 A \ldots)$*, then:*

- *Linearity forces* $C(A)$ *to be linear in* A.
- *Leibniz forces* $C(A)$ *to have no derivatives of* A.

Therefore, the correction must be a multiplicative term: matrix multiplication of A *by some object* Γ.

- *Given what we want to measure, we see why* $\Gamma^\alpha{}_{\beta\mu}$ ***must have three indices****: it measures how much* A^α *changes when we change* x^β *a little, in terms of the components of* A^μ *contracted against* $\Gamma^\alpha{}_{\beta\mu}$ *at the home point* x^μ*! Parallel transport simply must look exactly like this, no more, no less. (a physicist's hand-wave; mathematicians want more!)*
- *TODO: Is* $\Gamma^\alpha{}_{\beta\mu}$ *symmetric in* β and μ*? HINT: don't rathole on this now!*

Connection Coefficients

In this context, $\Gamma^\alpha{}_{\beta\mu}$ are called ***connection coefficients*** in the linear term $A.\Gamma$, or just ***connections***. Later, we'll use the same symbol $\Gamma^\alpha{}_{\beta\mu}$ for Christoffel Symbols, which are particular connection coefficients, those that satisfy restrictive physical requirements for general relativity.

- *We shall see, over the course of future notebooks, that* ***connections produce forces*** *in all of physics, classical and quantum! Existence of forces depends* ***both*** *on coordinate (gauge) choices and on intrinsic properties like mass and charge.*
- *In restricting connection coefficients for gravitation in general relativity, we're saying "only certain kinds of manifold geometries and topologies are useful for describing gravitation." Connection coefficients are more general than Christoffel symbols, pertaining to a broader class of manifolds.*

Covariant Derivative and Connections in $\mathcal{UD}$

$\mathcal{UD}$'s CD functions shows off the evaluated covariant derivative:

```
In[•]:= Echo[TensorForm[CD[A[𝒰[α]], x[𝒰[β]]]], "A^α'_;β' ="];
```

» $A^{\alpha'}{}_{;\beta'} = A^{\alpha}{}_{,\beta} + \left(A^{\lambda}\right) \Gamma^{\alpha}_{\beta\lambda}$

CD's held form exhibits semicolon notation!

```
In[•]:= Echo[HoldForm[CD[A[𝒰[α]], x[𝒰[β]]]], "A^α'_;β' ==="];
```

» $A^{\alpha'}{}_{;\beta'} === A^{\alpha}{}_{;\beta}$

Solving for Connection Coefficients $\Gamma^\alpha{}_{\beta\mu}$

Let's have $\mathcal{UD}$ do all the dirty work. We're not only going to solve for $\Gamma^\alpha{}_{\beta\mu}$, assuming only that $A^\alpha{}_{;\beta} \equiv (A^\alpha{}_{,\beta} + A^\mu \Gamma^\alpha{}_{\beta\mu})$ is tensorial (already encoded in $\mathcal{UD}$'s **CD**), but we'll derive the transformation law for $\Gamma^\alpha{}_{\beta\mu}$, proving that Γ is not tensorial, and show how the contraction $A^\mu \Gamma^\alpha{}_{\beta\mu}$ subtracts, exactly, the non-tensorial parts of the messy partial derivative $A^\alpha{}_{,\beta}$.

IMPORTANT: The final step invokes a Quotient Theorem, by which, schematically, $(T' \Gamma^{\alpha'}{}_{\beta'\mu'} = \langle Jacobians \rangle * T \Gamma^{\alpha}{}_{\beta\mu})$ implies $(\Gamma^{\alpha'}{}_{\beta'\mu'} = \langle Jacobians \rangle * \Gamma^{\alpha}{}_{\beta\mu})$, factoring out (quotienting) tensors T and T' and their Jacobians, if and only if (iff) they're truly arbitrary.

- *Lovelock & Rund write dire warnings about these theorems, that misapplication is the occasion of many published errors. Here, because we know the answer is correct, even by comparison to Lovelock & Rund, we're going to hand-wave the issue, here.*

As before, we'll write primed indices in a brutal way as ap and bp, and prettify them on the way out.

```
In[•]:= ClearAll[ap, bp, mu, nu, manualChainRule,
          allRules, targetTensor, Aprimed, xprimed, pretty];
        pretty = {ap → α', bp → β', cp → γ', mu → μ, nu → ν};
```

The Transformed Tensor We Want

Have $\mathcal{UD}$ write the form we're looking for, unsimplified as of yet. In the next steps, we'll do a round-about calculation to isolate the $A.\Gamma$ term and figure out how it must transform:

```
In[•]:= Echo[(targetTensor =
              (*Jacobians*)
             Partials[x[𝒰[ap]], x[𝒰[mu]]] *
              Partials[x[𝒰[nu]], x[𝒰[bp]]] *
              CD[A[𝒰[mu]], x[𝒰[nu]]]) /. pretty // TensorForm,
          "𝒰𝒟 says the tensor we want, A^α'_;β' ="];
```

» $\mathcal{UD}$ says the tensor we want, $A^{\alpha'}{}_{;\beta'} = \frac{\partial x^{\nu}}{\partial x^{\beta'}} \frac{\partial x^{\alpha'}}{\partial x^{\mu}} \left(A^{\mu}{}_{,\nu} + \left(A^{\lambda}\right) \Gamma^{\mu}_{\nu\lambda}\right)$

Calculate How Γ Must Transform

Convenience Variables

$\mathcal{UD}$'s `robustTransformRules` works on the primed, not differentiated tensor. Define a couple of convenience variables:

```
In[•]:= Echo[(Aprimed = A[𝒰[ap]] /. robustTransformRules) /. pretty // TensorForm, "A^α' ="];
        xprimed = x[𝒰[bp]];
```

» $A^{\alpha'} = \left(A^{\lambda}\right) \frac{\partial x^{\alpha'}}{\partial x^{\lambda}}$

Prepare a Custom Chain Rule

Insert a Jacobian for a primed `Partial` across coordinate system, i.e., rewrite $A^{\alpha}{}_{,\beta'}$ as $\frac{\partial x^{\mu}}{\partial x^{\beta'}} A^{\alpha}{}_{,\mu}$. We can't use $\mathcal{UD}$'s general robust transform rules because they would chain on the Jacobian, so we stop that manually. Also, because this is a special-purpose rule just for this calculation, we don't mind hard-coding on the primed index β':

```
In[•]:= (* Condition !MatchQ[expr,x[_]] prevents recursing a Jacobian on tself *)
manualChainRule =
  Partials[expr_, x[𝒰[bp]]] /; ! MatchQ[expr, x[_]] :>
   Module[{fresh = Unique["μ"]},
    Partials[x[𝒰[fresh]], x[𝒰[bp]]] *
     Partials[expr, x[𝒰[fresh]]]];
Echo[
  Partials[A[𝒰[α]], x[𝒰[bp]]] /. manualChainRule /. pretty // TensorForm, "A^α_,β' ="];
```

» $A^{\alpha}{}_{,\beta'} = \left(A^{\alpha}{}_{,\lambda}\right) \frac{\partial x^{\lambda}}{\partial x^{\beta'}}$

Combine $\mathcal{UD}$'s general differentiation rules with this special rule, showing nothing broke:

```
In[•]:= allRules = Flatten[{differentiationRules, manualChainRule}];
Echo[Partials[A[𝒰[α]], x[𝒰[bp]]] /. allRules /. pretty // TensorForm, "A^α_,β' ="];
```

» $A^{\alpha}{}_{,\beta'} = \left(A^{\alpha}{}_{,\lambda}\right) \frac{\partial x^{\lambda}}{\partial x^{\beta'}}$

Fetch the Broken Partial

Just set a variable to the broken partial we already know about:

```
In[•]:= Echo[(brokenPartialOnly = Partials[Aprimed, xprimed] //. allRules) /. pretty //
    TensorForm, "Broken partial A^α', β' ="];
```

» Broken partial $A^{\alpha'}{}_{,\beta'} = \left(A^{\lambda}\right) \frac{\partial^2 x^{\alpha'}}{\partial x^{\beta'} \partial x^{\lambda}} + \left(A^{\lambda}{}_{,\kappa}\right) \frac{\partial x^{\kappa}}{\partial x^{\beta'}} \frac{\partial x^{\alpha'}}{\partial x^{\lambda}}$

Get the Residual

Isolate the $A.\Gamma$ term we want from the target tensor minus the broken partial, i.e., calculate

$$A^{\mu'} \Gamma^{\alpha'}{}_{\beta'\mu'} = A^{\alpha'}{}_{;\beta'} - A^{\alpha'}{}_{,\beta'} \tag{9}$$

```
In[•]:= Echo[(gammaPrimeTimesVector = targetTensor - brokenPartialOnly) /. pretty //
    TensorForm, "A.Γ term :"];
```

» A.Γ term : $-\left(A^{\lambda}\right) \frac{\partial^2 x^{\alpha'}}{\partial x^{\beta'} \partial x^{\lambda}} - \left(A^{\lambda}{}_{,\kappa}\right) \frac{\partial x^{\kappa}}{\partial x^{\beta'}} \frac{\partial x^{\alpha'}}{\partial x^{\lambda}} + \frac{\partial x^{\nu}}{\partial x^{\beta'}} \frac{\partial x^{\alpha'}}{\partial x^{\mu}} \left(A^{\mu}{}_{,\nu} + \left(A^{\lambda}\right) \Gamma^{\mu}_{\nu\lambda}\right)$

Coalesce and Simplify (Magic)

This is messy, of the form $-A.\text{Hessian} - A\,(\text{primed}) + (\nabla A = \partial A + A.\Gamma)\,(\text{primed})$, but we can spot that the derivative terms $A^{\lambda}{}_{,\kappa}$ might cancel, if …

Here is the magic: if we collect the A' terms, $\mathcal{UD}$'s `TensorForm` expands, canonicalizes, and canceling the derivative term $A^{\lambda}{}_{,\kappa}$:

Transformation Law 1

In[•]:=
```
Echo[TensorForm[Collect[gammaPrimeTimesVector, A[𝒰[_]]] /. pretty], "A^λ' Γ^α'_β'λ' ="];
```

» $A^{\lambda'}\,\Gamma^{\alpha'}{}_{\beta'\lambda'} = -\left(A^{\ \lambda}\right)\frac{\partial^2 x^{\alpha'}}{\partial x^{\beta'}\,\partial x^{\lambda}} + \left(A^{\ \lambda}\right)\frac{\partial x^{\kappa}}{\partial x^{\beta'}}\frac{\partial x^{\alpha'}}{\partial x^{\rho}}\,\Gamma^{\rho}_{\kappa\lambda}$

The transformation law implies that Γ is not tensorial: **the *A*.Hessian term remains with a minus sign that exactly cancels out the Hessian from the messy partials**, by construction.

Quotient out *A*

Because *A* is an arbitrary contravariant vector, we can factor (or quotient) it out *A*.

Look again at the *A*.Γ term and its indices

In[•]:=
```
Echo[gammaPrimeTimesVector /. pretty, "A.Γ term (again) :"];
```

» A.Γ term (again) : $-\left(A^{\ \mu 15}\right)\frac{\partial^2 x^{\alpha'}}{\partial x^{\beta'}\,\partial x^{\mu 15}} -$

$\left(A^{\ \mu 15}{}_{,\mu 18}\right)\frac{\partial x^{\mu 18}}{\partial x^{\beta'}}\frac{\partial x^{\alpha'}}{\partial x^{\mu 15}} + \frac{\partial x^{\nu}}{\partial x^{\beta'}}\frac{\partial x^{\alpha'}}{\partial x^{\mu}}\left(A^{\ \mu}{}_{,\nu} + \left(A^{\ \lambda 14}\right)\Gamma^{\mu}_{\nu\lambda 14}\right)$

Temporarily substitute a known primed index, cp = γ', for an unknown Unique dummy, *μXY*

In[•]:=
```
(rhsWithTargetA = gammaPrimeTimesVector /. {A[𝒰[idx_]] :>
      Partials[x[𝒰[idx]], x[𝒰[cp]]] * A[𝒰[cp]]}) /. pretty // TensorForm
```

Out[•]=

$$-\left(A^{\ \lambda}\right)\frac{\partial^2 x^{\alpha'}}{\partial x^{\beta'}\,\partial x^{\kappa}}\frac{\partial x^{\kappa}}{\partial x^{\lambda}} - \left(\left(A^{\ \lambda}\right)\frac{\partial x^{\kappa}}{\partial x^{\lambda}}\right)_{,\rho}\frac{\partial x^{\rho}}{\partial x^{\beta'}}\frac{\partial x^{\alpha'}}{\partial x^{\kappa}} + \frac{\partial x^{\nu}}{\partial x^{\beta'}}\frac{\partial x^{\alpha'}}{\partial x^{\mu}}\left(\left(\left(A^{\ \gamma'}\right)\frac{\partial x^{\mu}}{\partial x^{\gamma'}}\right)_{,\nu} + \left(A^{\ \gamma'}\right)\frac{\partial x^{\lambda}}{\partial x^{\gamma'}}\,\Gamma^{\mu}_{\nu\lambda}\right)$$

Notice that some harmless Kronecker deltas, in the form of $\partial x^{\kappa}/\partial x^{\lambda}$ and $\partial x^{\kappa}/\partial x^{c}$, have been inserted.

Just collect the coefficients of $A^{\gamma'}$ and we're done

Final Transformation Law

In[•]:=
```
Echo[Coefficient[rhsWithTargetA, A[𝒰[cp]]] /. pretty // TensorForm, "Γ^α'_β'γ' ="];
```

» $\Gamma^{\alpha'}{}_{\beta'\gamma'} = -\frac{\partial^2 x^{\alpha'}}{\partial x^{\beta'}\,\partial x^{\lambda}}\frac{\partial x^{\lambda}}{\partial x^{\gamma'}} + \frac{\partial x^{\kappa}}{\partial x^{\beta'}}\frac{\partial x^{\lambda}}{\partial x^{\gamma'}}\frac{\partial x^{\alpha'}}{\partial x^{\rho}}\,\Gamma^{\rho}_{\kappa\lambda}$

Christoffel Symbols

Up to this point, we have not actually used a metric tensor! MTW devote seven whole chapters to differential geometry without a metric (9-15), stressing that many general results in differential geome-

try—the topological results—can be derived without the metric. In particular, the covariant derivative and the connection coefficients do not, actually, depend on any metrical ability to measure distances, but only on assumptions of continuity and differentiability.

But gravitation via general relativity requires a special case: Riemannian Geometry. For that, we need a metric, and even a bit more: symmetry conditions on both the connection coefficients and on the metric. From these additions, we derive the ultra-famous gravitational Christoffel symbols:

$$\Gamma^{\sigma}{}_{\mu\nu} = \frac{1}{2}\left(g^{\sigma\lambda}\right)\left(g_{\lambda\mu,\nu} + g_{\lambda\nu,\mu} - g_{\mu\nu,\lambda}\right) \tag{10}$$

This is equation 8.24 of MTW, after renaming some indices. It is also called “The Fundamental Theorem of Riemannian Geometry” in mathematical texts like do Carmo and Lee (see References).

Let’s do a step-by-step derivation via $\mathcal{UD}$.

Start by reserving some symbols.

In[•]:=
```
ClearAll[g, rSym, gSym, term1, term2, term3, koszul, targetIndex];
```

Physical Constraints

Torsion-Free Geometry

By MTW, Box 10.2, page 250, Γ is symmetric in its covariant (down) indices:

$$\Gamma^{\alpha}{}_{\beta\gamma} = \Gamma^{\alpha}{}_{\gamma\beta} \tag{11}$$

In[•]:=
```
Echo[rSym =
   r[u_, 𝒟[b_], 𝒟[c_]] :↦
    r[u, 𝒟[Sort[{b, c}][[1]]], 𝒟[Sort[{b, c}][[2]]]],
  "r Symmetry: "];
```

» Γ Symmetry: `r[u_, 𝒟[b_], 𝒟[c_]] :↦ r[u, 𝒟[Sort[{b, c}][[1]]], 𝒟[Sort[{b, c}][[2]]]]`

Metric Symmetry

By MTW, Equation 8.24, page 210, the metric g is symmetric (the display is a bit off, having gone through $\mathcal{UD}$ formatting, which was not designed for such)

In[•]:=
```
Echo[gSym =
   g[𝒟[a_], 𝒟[b_]] :↦
    g[𝒟[Sort[{a, b}][[1]]], 𝒟[Sort[{a, b}][[2]]]],
  "g Symmetry: "];
```

» g Symmetry: `g a_ b_ :↦ g a b`

Metric Compatibility Postulates

By MTW, Equation 8.23 and Chapter 14:

$$\nabla_{\lambda} g_{\mu\nu} = \nabla_{\mu} g_{\nu\lambda} = \nabla_{\nu} g_{\lambda\mu} = 0 \tag{12}$$

```
In[•]:= Echo[
          (term1 = CD[g[𝒟[μ], 𝒟[ν]], x[𝒰[λ]]] /. ΓSym /. gSym) // TensorForm, "0 = ∇λgμν =="];
```

» 0 = $\nabla_\lambda g_{\mu\nu}$ = $g_{\mu\nu,\lambda} - (g_{\kappa\nu})\,\Gamma^{\kappa}_{\lambda\mu} - (g_{\kappa\mu})\,\Gamma^{\kappa}_{\lambda\nu}$

```
In[•]:= Echo[
          (term2 = CD[g[𝒟[ν], 𝒟[λ]], x[𝒰[μ]]] /. ΓSym /. gSym) // TensorForm, "0 = ∇μgνλ =="];
```

» 0 = $\nabla_\mu g_{\nu\lambda}$ = $g_{\lambda\nu,\mu} - (g_{\kappa\nu})\,\Gamma^{\kappa}_{\lambda\mu} - (g_{\lambda\kappa})\,\Gamma^{\kappa}_{\mu\nu}$

```
In[•]:= Echo[
          (term3 = CD[g[𝒟[λ], 𝒟[μ]], x[𝒰[ν]]] /. ΓSym /. gSym) // TensorForm, "0 = ∇νgλμ =="];
```

» 0 = $\nabla_\nu g_{\lambda\mu}$ = $g_{\lambda\mu,\nu} - (g_{\kappa\mu})\,\Gamma^{\kappa}_{\lambda\nu} - (g_{\lambda\kappa})\,\Gamma^{\kappa}_{\mu\nu}$

Koszul Cyclic Permutation

By MTW, Equation 8.24b, page 210, find a linear combination of covariant derivatives of g that isolates a factor of Γ in a coordinate basis ($c_{abc} \equiv 0$). The particular one we choose is

$$\nabla_\mu g_{\nu\lambda} + \nabla_\nu g_{\lambda\mu} - \nabla_\lambda g_{\mu\nu} = 0 \tag{13}$$

```
In[•]:= Echo[koszul = (term2 + term3 - term1), "0 =="];
```

» 0 = $g_{\lambda\mu,\nu} + g_{\lambda\nu,\mu} - g_{\mu\nu,\lambda} + (g_{\lambda 19\,\nu})\,\Gamma^{\lambda 19}_{\lambda\mu} + (g_{\lambda 20\,\mu})\,\Gamma^{\lambda 20}_{\lambda\nu} -$
$(g_{\lambda\,\lambda 21})\,\Gamma^{\lambda 21}_{\mu\nu} - (g_{\lambda 22\,\nu})\,\Gamma^{\lambda 22}_{\lambda\mu} - (g_{\lambda 23\,\mu})\,\Gamma^{\lambda 23}_{\lambda\nu} - (g_{\lambda\,\lambda 24})\,\Gamma^{\lambda 24}_{\mu\nu}$

Solve for Γ

Because the Koszul equation is zero, we'll coalesce the terms with Γ and set them equal to the negative of the terms without Γ:

Pick the terms with a factor of Γ:

```
In[•]:= Echo[gammaPart = Select[List @@ koszul, ! FreeQ[#, Γ] &] // Total, "Γ part =="];
```

» Γ part = $(g_{\lambda 19\,\nu})\,\Gamma^{\lambda 19}_{\lambda\mu} + (g_{\lambda 20\,\mu})\,\Gamma^{\lambda 20}_{\lambda\nu} - (g_{\lambda\,\lambda 21})\,\Gamma^{\lambda 21}_{\mu\nu} - (g_{\lambda 22\,\nu})\,\Gamma^{\lambda 22}_{\lambda\mu} - (g_{\lambda 23\,\mu})\,\Gamma^{\lambda 23}_{\lambda\nu} - (g_{\lambda\,\lambda 24})\,\Gamma^{\lambda 24}_{\mu\nu}$

Coalesce (canonicalize) and simplify:

```
In[•]:= Echo[lhsRaw = -gammaPart // CanonicalizeIndices, "Γ part (canonical) =="];
```

» Γ part (canonical) = $2\,(g_{\lambda\,i1})\,\Gamma^{i1}_{\mu\nu}$

Pick out the terms with no Γ factor:

```
In[•]:= Echo[rhsRaw = Select[List @@ koszul, FreeQ[#, Γ] &] // Total, "g partials =="];
```

» g partials = $g_{\lambda\mu,\nu} + g_{\lambda\nu,\mu} - g_{\mu\nu,\lambda}$

```
In[•]:= Echo[TensorForm[lhsRaw == rhsRaw], "Isolated Equation :"];
```

» Isolated Equation : $2\,(g_{\lambda\kappa})\,\Gamma^{\kappa}_{\mu\nu} == g_{\lambda\mu,\nu} + g_{\lambda\nu,\mu} - g_{\mu\nu,\lambda}$

Isolate Γ

The strategy is to contract this equation over a contravariant metric, schematically $g^{\sigma\lambda}$, to generate a Kronecker delta, $\delta^{\sigma}{}_{\rho}$, leaving a bare Γ on the left. It takes a few steps of $\mathcal{UD}$ algebra to do this robustly:

Find the metric inside the lhs:

```
In[•]:= Echo[metricFound = First[Cases[lhsRaw, g[__], Infinity]], "g found :"];
```

» g found : $g_{\lambda\, i1}$

Find the Γ inside the lhs:

```
In[•]:= Echo[gammaFound = First[Cases[lhsRaw, Γ[__], Infinity]], "Γ found :"];
```

» Γ found : $\Gamma^{i1}_{\mu\,\nu}$

Extract raw symbols from the indices:

```
In[•]:= Echo[metricIndices = First /@ (List @@ metricFound), "g indices :"];
        Echo[gammaUpIndex = First[First[gammaFound]], "Γ up index"];
```

» g indices : {λ, i1}

» Γ up index i1

The *g* index that matches the Γ up-index is the dummy; the other is the target index:

```
In[•]:= Echo[targetIndex = If[metricIndices[[1]] === gammaUpIndex,
            metricIndices[[2]], metricIndices[[1]]], "target index :"];
```

» target index : λ

Construct the specific contravariant *g* needed; include a factor of 1/2 to cancel out the factor of 2 from the Γ part

```
In[•]:= Echo[invMetricFactor = 1/2 g[𝒰[σ], 𝒰[targetIndex]], "1/2 g^-1 ="];
```

» $\frac{1}{2} g^{-1} = \frac{1}{2} \left(g^{\sigma\ \lambda}\right)$

Contract via a custom rule:

```
In[•]:= Echo[deltaContractionRule = δ[𝒰[s_], 𝒟[r_]] * Γ[𝒰[r_], a_, b_] :> Γ[𝒰[s], a, b],
          "δ-Γ contraction rule: "];
```

» δ–Γ contraction rule: $\Gamma[\mathcal{U}[r\_], a\_, b\_]\ \delta^{s}_{r\_} :\to \Gamma[\mathcal{U}[s], a, b]$

- *TODO: promote the contraction rule to the main 𝒰𝒟 code base.*

Solve the lhs:

```
In[•]:= Echo[lhsSolved = lhsRaw * invMetricFactor //. metricRules //. deltaContractionRule,
          "LHS solved :"];
```

» LHS solved : $\Gamma^{\sigma}_{\mu\,\nu}$

Rewrite the RHS with the contravariant metric:

```
In[•]:= Echo[rhsSolved = Collect[rhsRaw * invMetricFactor, g[𝒰[_], 𝒰[_]]], "RHS solved :"];
```

» RHS solved : $\frac{1}{2} \left(g^{\sigma\ \lambda}\right) \left(g_{\lambda\ \mu,\nu} + g_{\lambda\ \nu,\mu} - g_{\mu\ \nu,\lambda}\right)$

Final Equation

```
In[•]:= Echo[(finalEquation = lhsSolved == rhsSolved) // TensorForm,
          "Fundamental Theorem of Differential Geometry :"];
        TensorForm[finalEquation]
```

» Fundamental Theorem of Differential Geometry : $\Gamma^{\sigma}_{\mu\nu} == \frac{1}{2}\left(g^{\sigma\lambda}\right)\left(g_{\lambda\mu,\nu} + g_{\lambda\nu,\mu} - g_{\mu\nu,\lambda}\right)$

Out[•]=

$$\Gamma^{\sigma}_{\mu\nu} == \frac{1}{2}\left(g^{\sigma\lambda}\right)\left(g_{\lambda\mu,\nu} + g_{\lambda\nu,\mu} - g_{\mu\nu,\lambda}\right)$$

Conclusion

We began this notebook with a non-tensorial partial derivative of a contravariant tensor. However, by adding a correction and *then* demanding linearity and tensoriality, we forced a covariant derivative to emerge, with the correction proportional to the non-tensorial connection Γ.

Also, not just accepting textbook derivations of the Christoffel symbols for gravitation, we derive them *ab-initio* in $\mathcal{UD}$. By insisting that lengths be preserved ($\nabla g = 0$) and twisting be forbidden ($\Gamma_{\mu\nu} = \Gamma_{\nu\mu}$), $\mathcal{UD}$ collapses the Koszul sum into the standard formula:

$$\Gamma^{\sigma}{}_{\mu\nu} = \frac{1}{2}\left(g^{\sigma\lambda}\right)\left(g_{\lambda\mu,\nu} + g_{\lambda\nu,\mu} - g_{\mu\nu,\lambda}\right)$$

The form of the Christoffel symbols shows that gravitation, manifested in the connection, is solely determined by the metric and its ordinary partial derivatives.

With a working covariant derivative in $\mathcal{UD}$'s Relativity Toolkit, we are equipped for future installments on geodesics, geodesic deviation, tidal forces, accelerated observers, Schwarzschild and Kerr metrics, black holes, etc.

References

1. Poisson, Eric. *A Relativist's Toolkit: The Mathematics of Black Hole Mechanics*. Cambridge University Press, 2004.
2. Beckman, Brian. *From the steam age to quantum fields: a unified approach via UD tensorial calculus*. Wolfram Community post.
3. Beckman, Brian. *General relativity in the UD calculus*. Wolfram Community post.
4. Beckman, Brian. *Formal Differential Geometry in the UD Calculus: Part 1*. Wolfram Community post.
5. [MTW] Misner, Charles W.; Thorne, Kip S.; Wheeler, John Archibald. *Gravitation*. Princeton University Press, 2017.
6. [L&R] Lovelock, David and Rund, Hanno. *Tensors, Differential Forms, and Variational Principles*. Dover.
7. John M. Lee, *Riemannian Manifolds: An Introduction to Curvature*. Springer.

8. Manfredo P. do Carmo, *Riemannian Geometry*. Springer.

9. Wolfram Function Repository. *MetricTensor, RiemannTensor, EinsteinTensor*. For component-based calculations.

10. Wolfram Research. *Mathematica Documentation.* Tensor Calculus.